

Practicum No.3**Date:.....****Address calculation using addressing modes****I. Practical Significance**

Effective Address (EA) calculation is a fundamental building block in computer architecture, instruction execution, memory access and compiler design. Addressing modes help processors access memory efficiently during instruction execution. They eliminate redundant instructions, reduce program size, accelerate array/pointer operations, and simplify loop processing. MATLAB simulation enables students to visualize and calculate effective addresses for multiple addressing modes including:

- Direct addressing
- Indirect addressing
- Indexed addressing
- Base + Index
- Base + Index + Displacement
- Scaled Index addressing
- Stack addressing
- PC-relative addressing

II. Competency and Industry-Specific Practical Skills

Use the principles of computer architecture to optimize system performance

- a) Select appropriate addressing mode for a specific memory operation (PDO)
- b) Translate an instruction into its effective address (CDO)
- c) Use MATLAB to validate address calculations systematically (PDO)

III. Relevant CO(s) *(As stated in the Course Structure)*

- **CO1.** Use the relevant instruction cycle to efficiently manage the processor's functionality.

IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

- **Develop** programs to do address calculation for the given addressing modes.

V. Related ADO(s) *(As stated in the Course Structure)*

- a) Follow ethical practices in designing or evaluating the system design.
- b) Apply systematic design principles and debugging techniques.
- c) Function effectively as a team member.

VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

An addressing mode defines how a processor locates data required by an instruction.

The Effective Address (EA) is the actual memory address from which data is fetched or stored.

Typical Addressing Modes & EA calculation rules:

Addressing Mode	EA Formula	Example
Immediate	Operand is part of instruction	MOV R1, #25
Direct	EA = Address	LOAD R1, 500
Indirect	EA = (Address)	LOAD R1, (500)
Register	Operand in register	ADD R1, R2
Register Indirect	EA = (Ri)	LOAD R1, (R2)
Indexed	EA = Base + Index	LOAD R1, A[Ri]
Displacement	EA = Base + Disp	LOAD R1, 100(R2)
Base + Index + Displacement	EA = Base + Index + Disp	A[R1 + R2 + 4]
Scaled Index	EA = Base + Index × Scale	Arrays, structures
PC-relative	EA = PC + Offset	Branch instructions
Stack addressing	EA = SP ± Offset	PUSH / POP

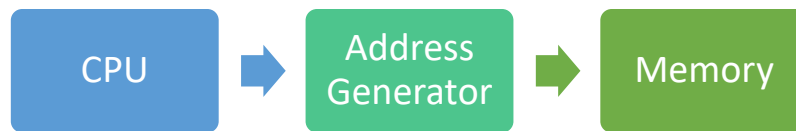
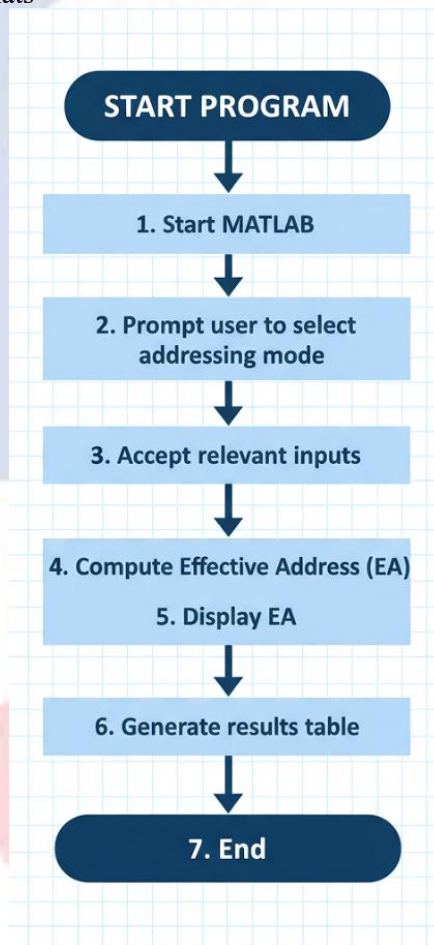


Figure 1: EA calculation block diagram

VII. Practicum Set-up/Circuit Diagram/Algorithm/Flowchart *(Include probable photos of experimental set-up.)*

Students must have:

- MATLAB installed
- Script file: exp3_addressing_modes.m
- Input dataset (base, index, displacement, PC, SP, scale)
- Example instruction formats



VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	MATLAB	(R2016b or newer) or GNU Octave (with mostly compatible syntax).	1 No.
2	PC	PC with MATLAB installed.	1 No.
3	Text editor / MATLAB Editor	-	1 No.

IX. Safety Precautions

- a) **Save** work frequently.
- b) **Comment** code and include header describing author, date, input format and usage.
- c) For shared code, **remove** hard-coded absolute paths.
- d) **Validate** user inputs to avoid invalid logical values and runtime errors.

X. Procedure *(Self-Instructional)*

- a) **Display** the list of supported addressing modes to the user.
- b) **Select** the addressing mode based on user input (Direct, Indirect, Indexed, Displacement, Scaled Index, PC-Relative, Stack).
- c) **Accept** the required input values such as Base, Index, Displacement, Scale factor, PC, or SP depending on the chosen mode.
- d) **Validate** the entered inputs for correct numeric format and applicable addressing mode requirements.
- e) **Compute** the Effective Address (EA) using the corresponding formula for the selected addressing mode along with memory access count, power consumption and time (Using tic, toc. tic and toc do not measure actual CPU execution time. They provide a relative software-level comparison that reflects the impact of memory access differences between addressing modes).
- f) **Print** the computed the addressing mode, computed Effective Address, memory access count, execution time, execution speed, and power consumption in the MATLAB command window.
- g) **Repeat** the calculation if the user selects another addressing mode.
- h) **Terminate** the program when the user chooses to exit.

Note: Instruction fetch is not considered while counting memory accesses; only operand access is considered. In indirect addressing, pointer dereferencing is assumed to focus on effective address computation.

```
clc;
clear;
```

```
choice = 1;
```

```
while choice ~= 0
```

```
    % ----- Display Menu -----
    disp('-----');
    disp('ADDRESSING MODES');
    disp('1. Immediate Addressing');
    disp('2. Register Addressing');
    disp('3. Direct Addressing');
    disp('4. Indirect Addressing');
    disp('5. Indexed Addressing');
    disp('6. Displacement Addressing');
    disp('7. Base + Index + Displacement Addressing');
    disp('8. Scaled Index Addressing');
    disp('9. PC-Relative Addressing');
    disp('10. Stack Addressing');
    disp('0. Exit');
    disp('-----');
```

```
choice = input('Enter your choice: ');

if choice == 0
    disp('Program terminated. ');
    break;
end

EA = 'Not Applicable';
mem_access = 0;
exec_time = 0;

switch choice

case 1 % Immediate Addressing
    disp('Immediate Addressing Selected');
    operand = input('Enter immediate operand value: ');

    tic;
    EA = 'Not Applicable';
    mem_access = 0;
    exec_time = toc;

case 2 % Register Addressing
    disp('Register Addressing Selected');
    regVal = input('Enter register value: ');

    tic;
    EA = 'Not Applicable';
    mem_access = 0;
    exec_time = toc;

case 3 % Direct Addressing
    disp('Direct Addressing Selected');
    addr = input('Enter memory address: ');

    tic;
    EA = addr;
    mem_access = 1;
    exec_time = toc;

case 4 % Indirect Addressing
    disp('Indirect Addressing Selected');
    ptr = input('Enter pointer value (address obtained from memory): ');

    tic;
    EA = ptr;          % Pointer dereference assumed
    mem_access = 2;    % Fetch pointer + operand
```

```
exec_time = toc;
```

case 5 % Indexed Addressing

```
disp('Indexed Addressing Selected');  
base = input('Enter Base address: ');  
index = input('Enter Index value: ');
```

```
tic;  
EA = base + index;  
mem_access = 1;  
exec_time = toc;
```

case 6 % Displacement Addressing

```
disp('Displacement Addressing Selected');  
base = input('Enter Base address: ');  
dispVal = input('Enter Displacement value: ');
```

```
tic;  
EA = base + dispVal;  
mem_access = 1;  
exec_time = toc;
```

case 7 % Base + Index + Displacement

```
disp('Base + Index + Displacement Addressing Selected');  
base = input('Enter Base address: ');  
index = input('Enter Index value: ');  
dispVal = input('Enter Displacement value: ');
```

```
tic;  
EA = base + index + dispVal;  
mem_access = 1;  
exec_time = toc;
```

case 8 % Scaled Index Addressing

```
disp('Scaled Index Addressing Selected');  
base = input('Enter Base address: ');  
index = input('Enter Index value: ');  
scale = input('Enter Scale factor: ');
```

```
tic;  
EA = base + (index * scale);  
mem_access = 1;  
exec_time = toc;
```

case 9 % PC-Relative Addressing

```
disp('PC-Relative Addressing Selected');  
PC = input('Enter Program Counter (PC) value: ');  
offset = input('Enter Offset value: ');
```

```

tic;
EA = PC + offset;
mem_access = 1;
exec_time = toc;

case 10 % Stack Addressing
    disp('Stack Addressing Selected');
    SP = input('Enter Stack Pointer (SP) value: ');

    tic;
    EA = SP - 1;      % Pre-decrement stack
    mem_access = 1;
    exec_time = toc;

otherwise
    disp('Invalid choice. Please select a valid option. ');
    continue;
end

% ----- Performance Classification -----
if mem_access == 0
    exec_speed = 'Fast';
    power = 'Less';
elseif mem_access == 1
    exec_speed = 'Moderate';
    power = 'More';
else
    exec_speed = 'Slow';
    power = 'More';
end

% ----- Display Results -----
disp('-----');
disp(['Effective Address (EA) : ', num2str(EA)]);
disp(['Memory Accesses      : ', num2str(mem_access)]);
disp(['Execution Time (sec)  : ', num2str(exec_time)]);
disp(['Execution Speed      : ', exec_speed]);
disp(['Power Consumption     : ', power]);
disp('-----');

end

```

Sample Output Table:

Mode	Input	Computed Effective Address (EA)
Immediate	Operand = 25	Not Applicable

Register	R1 = 500	Not Applicable
Direct	Address = 1200	1200
Indirect	Pointer Value = 1500	1500
Indexed	Base = 1000, Index = 20	1020
Displacement	Base = 1000, Displacement = 24	1024
Base + Index + Displacement	Base = 1000, Index = 20, Disp = 4	1024
Scaled Index	Base = 2000, Index = 3, Scale = 4	2012
PC-Relative	PC = 5000, Offset = -10	4990
Stack	SP = 3000	2999

```

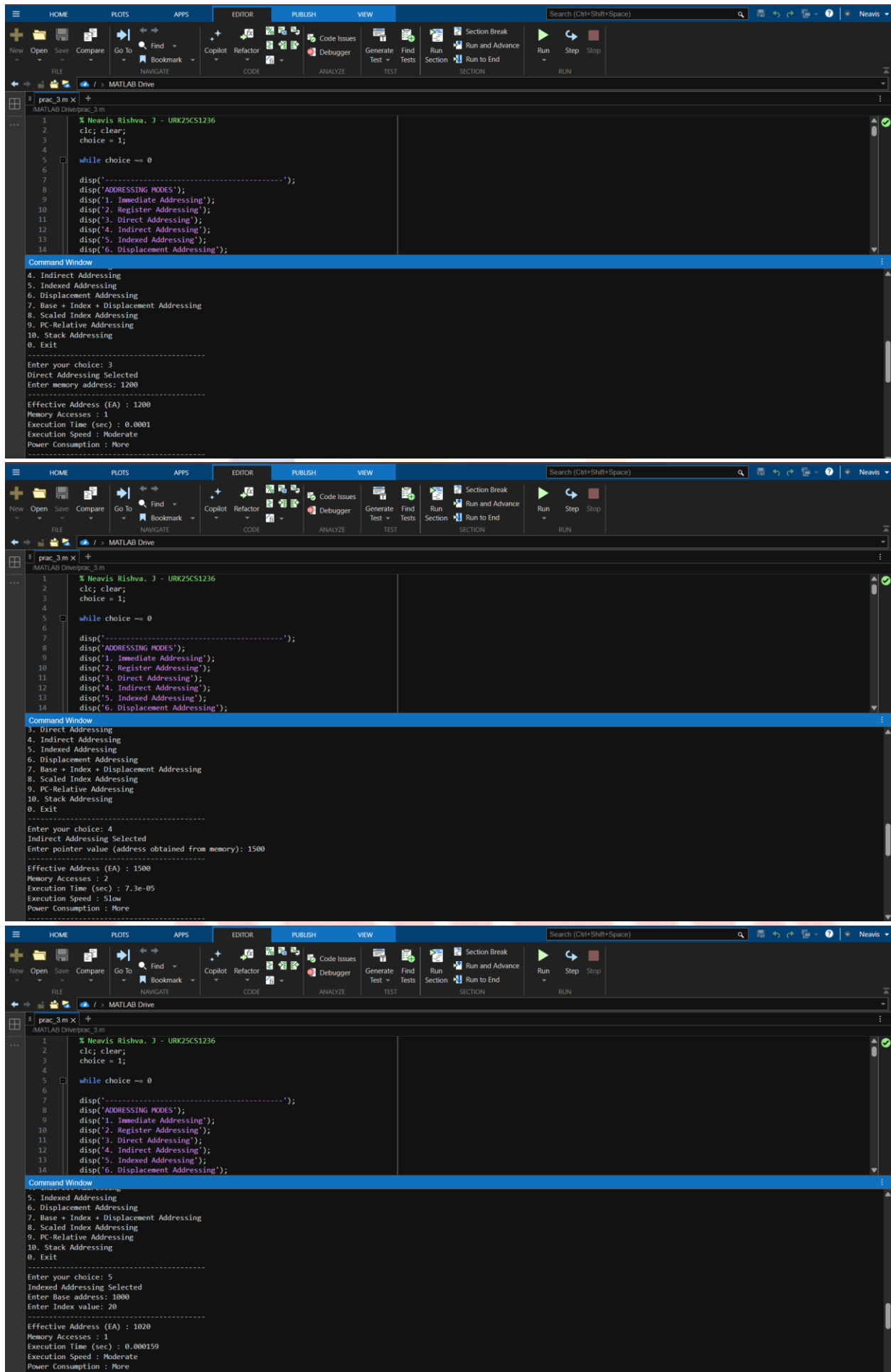
1 % Neavis Rishva. J - URK25CS1236
2 clc; clear;
3 choice = 1;
4
5 while choice ~= 0
6
7     disp('-----');
8     disp('ADDRESSING MODES');
9     disp('1. Immediate Addressing');
10    disp('2. Register Addressing');
11    disp('3. Direct Addressing');
12    disp('4. Indirect Addressing');
13    disp('5. Indexed Addressing');
14    disp('6. Displacement Addressing');
15    disp('7. Base + Index + Displacement Addressing');
16    disp('8. Scaled Index Addressing');
17    disp('9. PC-Relative Addressing');
18
19 Command Window
20 ADDRESSING MODES
21 1. Immediate Addressing
22 2. Register Addressing
23 3. Direct Addressing
24 4. Indirect Addressing
25 5. Indexed Addressing
26 6. Displacement Addressing
27 7. Base + Index + Displacement Addressing
28 8. Scaled Index Addressing
29 9. PC-Relative Addressing
30 10. Stack Addressing
31 0. Exit
32 -----
33 Enter your choice: 1
34 Immediate Addressing Selected
35 Enter immediate operand value: 25

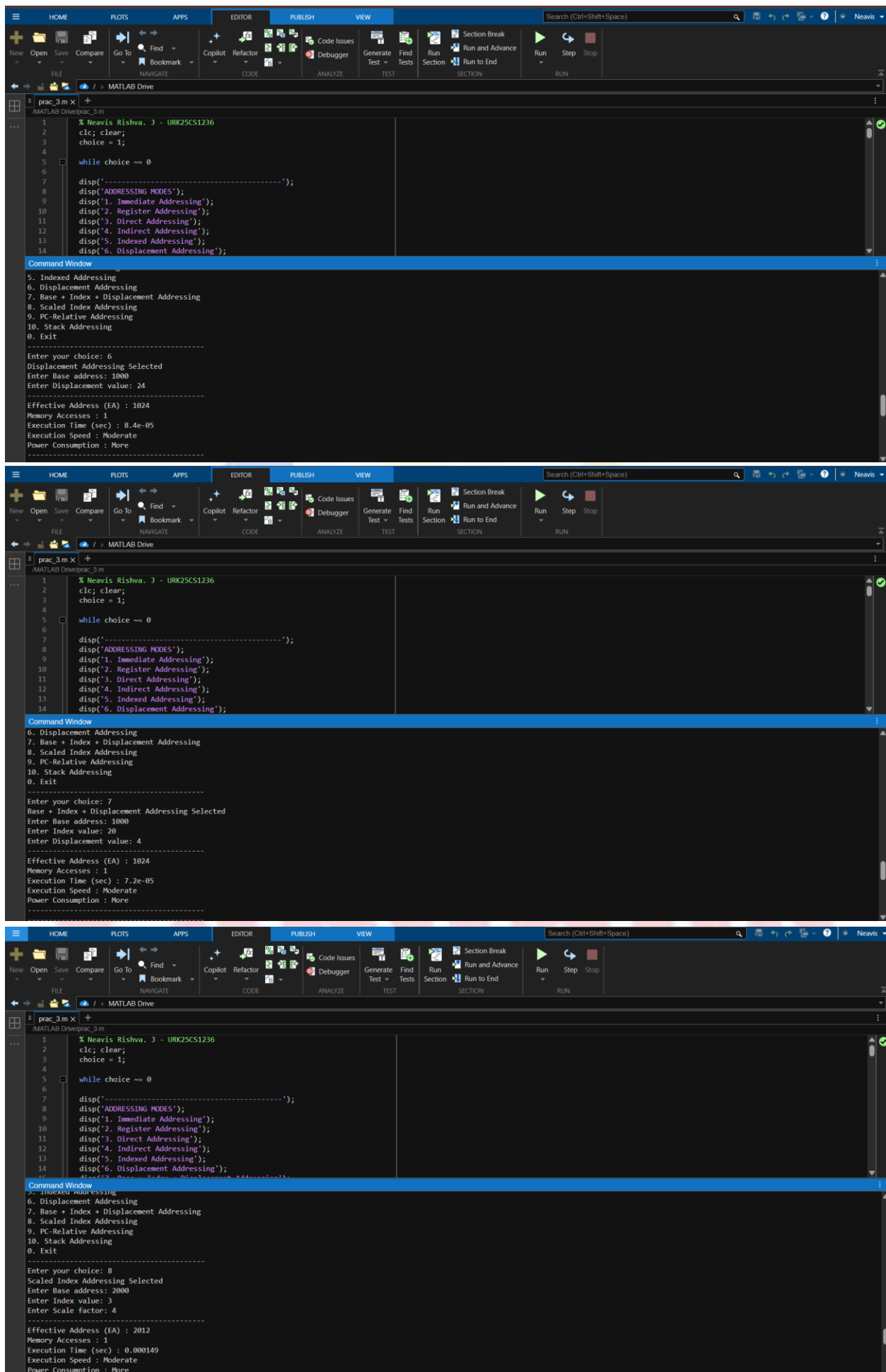
```

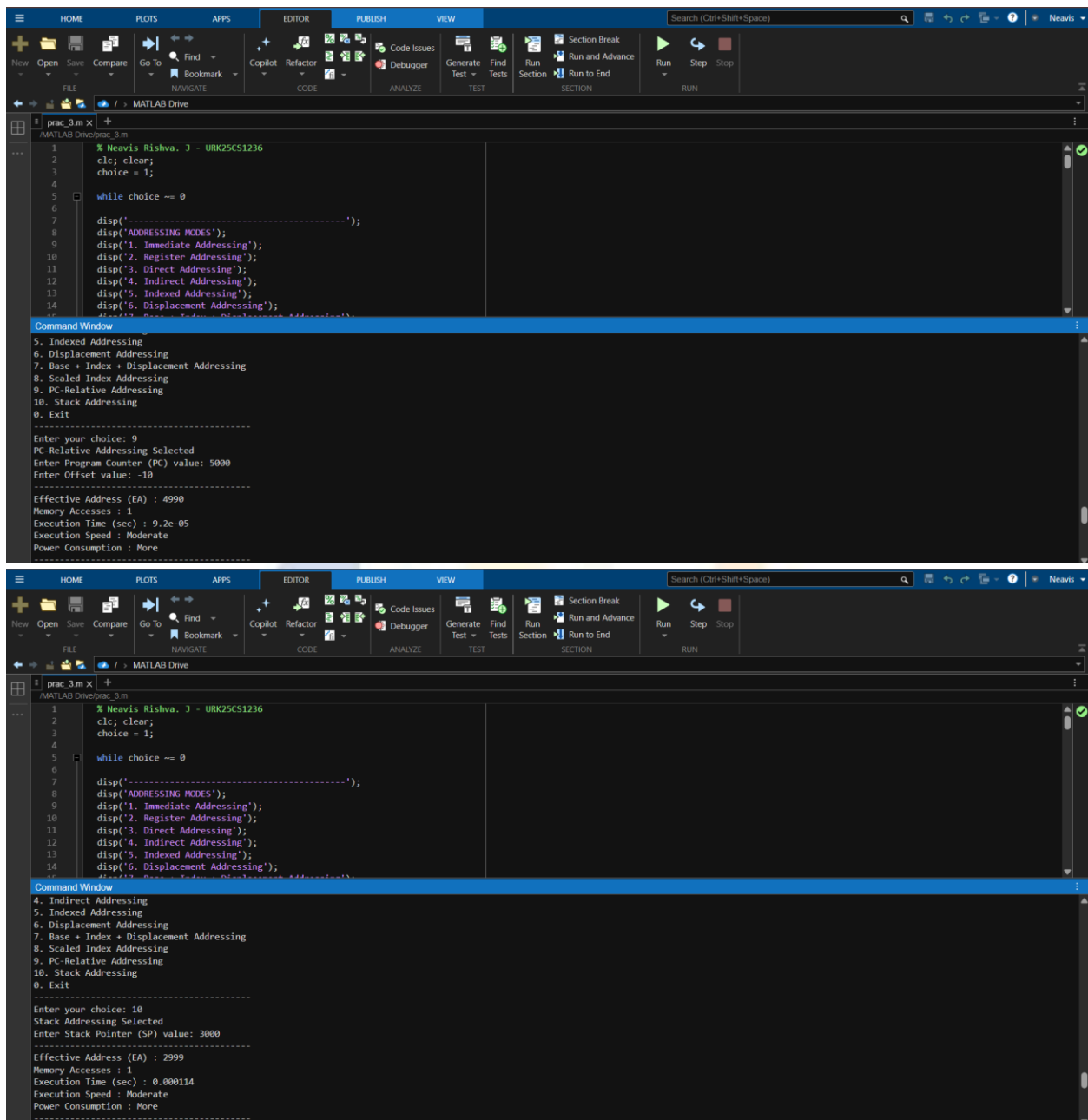
```

1 % Neavis Rishva. J - URK25CS1236
2 clc; clear;
3 choice = 1;
4
5 while choice ~= 0
6
7     disp('-----');
8     disp('ADDRESSING MODES');
9     disp('1. Immediate Addressing');
10    disp('2. Register Addressing');
11    disp('3. Direct Addressing');
12    disp('4. Indirect Addressing');
13    disp('5. Indexed Addressing');
14    disp('6. Displacement Addressing');
15
16 Command Window
17 4. Indirect Addressing
18 5. Indexed Addressing
19 6. Displacement Addressing
20 7. Base + Index + Displacement Addressing
21 8. Scaled Index Addressing
22 9. PC-Relative Addressing
23 10. Stack Addressing
24 0. Exit
25 -----
26 Enter your choice: 2
27 Register Addressing Selected
28 Enter register value: 500
29 -----
30 Effective Address (EA) : Not Applicable
31 Memory Accesses : 0
32 Execution Time (sec) : 0.000152
33 Execution Speed : Fast
34 Power Consumption : Less
35 -----

```







XI. Actual Resources Used *(To be filled by the students)*

S. No.	Name of Resource	Broad Specifications		Quantity	Remarks (If any)
		Make	Details		
1.	MATLAB	Version R2016b	1	-	MATLAB
2.	Laptop	Intel i5	1	-	Laptop
3.	MATLAB Editor	N/A	1	-	MATLAB Editor

XII. Actual Procedure Followed *(To be filled by the students)*

1. MATLAB software was opened on the PC and the script file for addressing mode calculation was created/loaded.
2. The program was executed to display the menu of available addressing modes.
3. An addressing mode was selected by entering the corresponding choice.

4. Required input values such as Base address, Index value, Displacement, PC, SP, or Scale factor were entered according to the selected mode.
5. The program computed the Effective Address (EA), number of memory accesses, execution time, execution speed, and power consumption.
6. The output results displayed in the MATLAB command window were observed and recorded in the observation table.
7. Steps 3–6 were repeated for different addressing modes to collect multiple test cases.
8. After completing the required runs, the program was terminated.

XIII. Additional Questions

1. Execute the MATLAB program using Immediate addressing and Direct addressing for the same operand value and compare the number of memory accesses, execution time, and suitability for real-time systems.

CODE:

```
%Neavis Rishva.J - URK25CS1236
```

```
clc; clear;
```

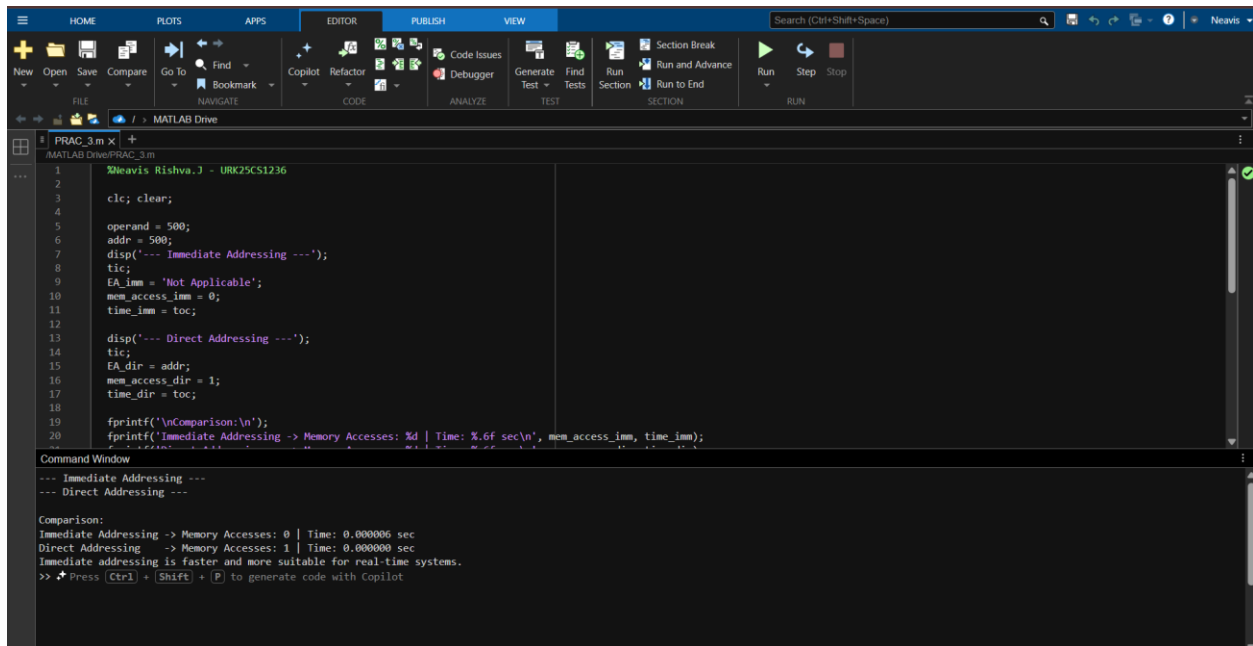
```
operand = 500;
addr = 500;
disp('--- Immediate Addressing ---');
tic;
EA_imm = 'Not Applicable';
mem_access_imm = 0;
time_imm = toc;
```

```
disp('--- Direct Addressing ---');
tic;
EA_dir = addr;
mem_access_dir = 1;
time_dir = toc;
```

```
fprintf('\nComparison:\n');
fprintf('Immediate Addressing -> Memory Accesses: %d | Time: %.6f sec\n', mem_access_imm,
time_imm);
fprintf('Direct Addressing -> Memory Accesses: %d | Time: %.6f sec\n', mem_access_dir,
time_dir);
```

```
if mem_access_imm < mem_access_dir
    disp('Immediate addressing is faster and more suitable for real-time systems.');
```

```
end
```



- Run the MATLAB program using Register addressing and Register Indirect addressing with the same register value and analyze the difference in memory access count and execution time.

CODE:

%Neavis Rishva. J – URK25CS1236

clc; clear;

regVal = 1000;

disp('--- Register Addressing ---');

tic;

EA_reg = 'Not Applicable';

mem_access_reg = 0;

time_reg = toc;

disp('--- Register Indirect Addressing ---');

tic;

EA_reg_ind = regVal;

mem_access_reg_ind = 1;

time_reg_ind = toc;

fprintf('\nComparison:\n');

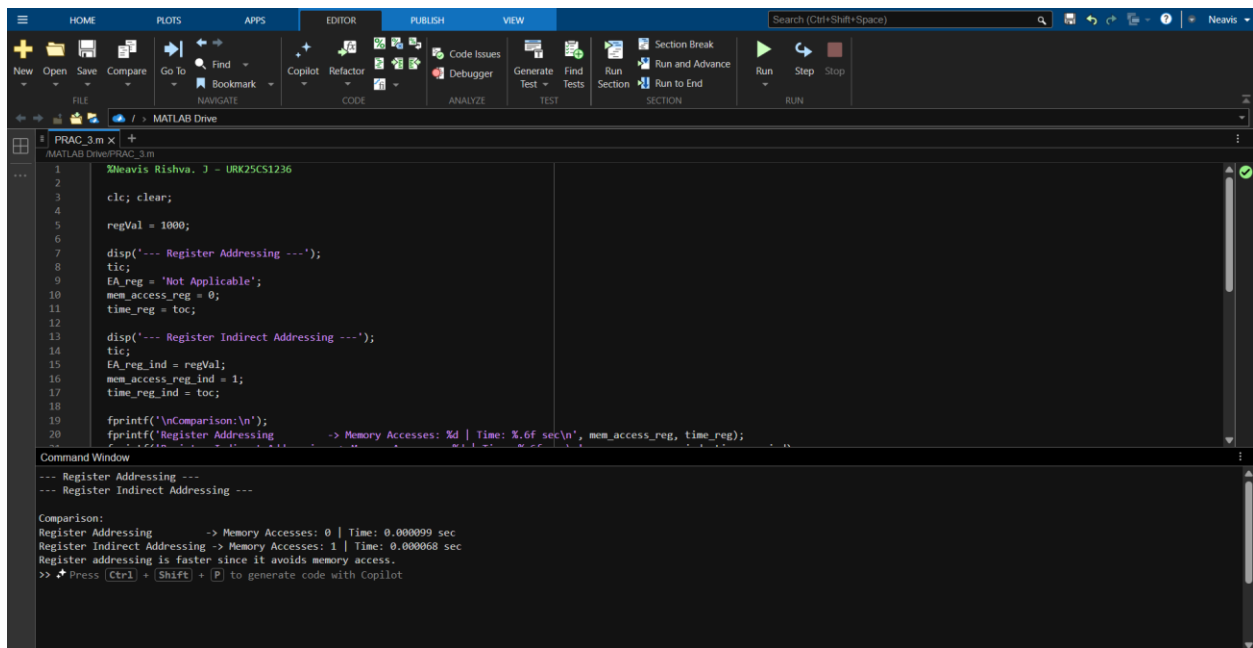
fprintf('Register Addressing -> Memory Accesses: %d | Time: %.6f sec\n', mem_access_reg, time_reg);

fprintf('Register Indirect Addressing -> Memory Accesses: %d | Time: %.6f sec\n', mem_access_reg_ind, time_reg_ind);

if mem_access_reg < mem_access_reg_ind

disp('Register addressing is faster since it avoids memory access.');

end



- Execute the program using Displacement addressing and Base + Index + Displacement addressing with the same base and offset values and analyze the effect of additional address computation on execution time.

CODE:

%Neavis Rishva. J – URK25CS1236

```
clc; clear;
```

```
base = 1000;
dispVal = 20;
index = 10;
```

```
disp('--- Displacement Addressing ---');
tic;
EA_disp = base + dispVal;
mem_access_disp = 1;
time_disp = toc;
```

```
disp('--- Base + Index + Displacement Addressing ---');
tic;
EA_bidxdisp = base + index + dispVal;
mem_access_bidxdisp = 1;
time_bidxdisp = toc;
```

```
fprintf('\nComparison:\n');
fprintf('Displacement Addressing          -> EA: %d | Time: %.6f sec\n', EA_disp, time_disp);
fprintf('Base + Index + Displacement Addressing -> EA: %d | Time: %.6f sec\n', EA_bidxdisp, time_bidxdisp);
```

```
if time_disp < time_bidxdisp
    disp('Displacement addressing is faster because it requires fewer arithmetic additions.');
```

```
end
```

```

1 %leavis Rishva. J - URK25CS1236
2
3 clc; clear;
4
5 base = 1000;
6 dispVal = 20;
7 index = 10;
8
9 disp('--- Displacement Addressing ---');
10 tic;
11 EA_disp = base + dispVal;
12 mem_access_disp = 1;
13 time_disp = toc;
14
15 disp('--- Base + Index + Displacement Addressing ---');
16 tic;
17 EA_bidxdisp = base + index + dispVal;
18 mem_access_bidxdisp = 1;
19 time_bidxdisp = toc;
20

```

Command Window

```

--- Displacement Addressing ---
--- Base + Index + Displacement Addressing ---

Comparison:
Displacement Addressing      -> EA: 1020 | Time: 0.000141 sec
Base + Index + Displacement Address -> EA: 1030 | Time: 0.000067 sec
>> Press [Ctrl] + [Shift] + [P] to generate code with Copilot

```

XIV. Observations and Dimensional Measurement *(To be filled by the students)*

S. No.	Test Case ID	Instruction / Addressing Mode Selected	Input Values (Base / Index / Disp / PC / SP)	Effective Address (EA)	No. of Memory Accesses	Execution Time (tic/toc)	Execution Speed (Fast / Moderate / Slow)	Power Consumption (Less/More)
1	TC-1	Immediate Addressing	Operand = 25	NA	0	0.000012	Fast	Less
2	TC-2	Indirect Addressing	Pointer = 1500	1500	2	0.000061	Slow	More
3	TC-3	Indexed Addressing	Base = 1000, Index = 20	1020	1	0.000030	Moderate	More
4	TC-4	PC-Relative Addressing	PC = 5000, Offset = -10	4990	1	0.000040	Moderate	More

Note: Instructions for Filling the Table:

- **Test Case ID:**
Assign a unique ID for each run (TC-1, TC-2, etc.).
- **Instruction / Addressing Mode Selected:**
Mention the addressing mode chosen in the MATLAB program (e.g., Direct, Indirect, Indexed, Displacement, Base+Index+Disp, PC-Relative, Stack).
- **Input Values:**
Enter the values provided by the user such as Base address, Index value, Displacement, PC, or SP depending on the addressing mode.
- **Effective Address (EA):**
Record the effective address calculated and displayed by the MATLAB program.
- **No. of Memory Accesses:**
Enter the number of memory accesses required for the selected addressing mode as shown by the program.
- **Execution Time (tic/toc):**
Record the execution time displayed by the MATLAB program (relative comparison).
- **Execution Speed:**
Mark **Fast** or **Slow** based on the number of memory accesses.
- **Power Consumption:**
Mark **Less** or **More** based on memory access involvement.

XV. Results/Observations

1. Immediate and Register addressing modes required **0 memory accesses**, resulting in the fastest execution time and lowest power consumption.
2. Direct, Indexed, Displacement, PC-relative, and Stack addressing modes required **1 memory access**, resulting in moderate execution time and moderate power consumption.
3. Indirect addressing required **2 memory accesses**, which resulted in slower execution speed and higher power consumption compared to other modes.
4. The Effective Address (EA) was correctly computed according to the addressing mode formulas

XVI. Interpretation of Results

1. Addressing modes that **do not access memory (Immediate and Register)** are faster because operands are available within the instruction or CPU register itself. This reduces latency and power consumption.
2. Addressing modes requiring **one memory access** show moderate performance because the processor must fetch data from memory once.
3. Indirect addressing is slower because it requires **two memory accesses** — one to fetch the pointer and another to fetch the operand — increasing delay and energy usage.
4. Complex address computation (Base + Index + Displacement) slightly increases execution time due to additional arithmetic operations, even though memory access count remains the same.

XVII. Conclusions

1. **Immediate and Register addressing modes are most suitable for real-time systems** due to zero memory access and faster execution.
2. **Indirect addressing is the slowest** because it involves multiple memory accesses.
3. Addressing modes like Indexed and Displacement are efficient for array and structured data access.
4. PC-relative addressing is suitable for control flow and relocatable code.
5. The experiment confirms that reducing memory access improves execution speed and lowers power consumption.

XVIII. Suggested Practicum Related Questions:

Note: Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.

1. **Justify** how the number of memory accesses differs between Direct and Indirect addressing modes and explain its impact on execution speed and power consumption.
2. **Determine** how Effective Address (EA) computation varies across Indexed, Displacement, and Base + Index + Displacement addressing modes using the MATLAB simulation results.
3. **Recommend** a suitable addressing mode for real-time systems by analyzing memory access count and execution time obtained using `tic` and `toc`.
4. **Generate** a comparative analysis using MATLAB outputs to show why Register or Immediate addressing modes provide better predictability than memory-based addressing modes.
5. **Justify** why PC-relative addressing is preferred over Direct addressing in modern processors for control-flow instructions and relocatable code.

XIX. References / Suggestions for further reading

1. Computer Organization and Architecture – Instruction Set Architecture and Addressing Modes
2. Processor Architecture Textbooks

XX Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the '*process-related skills/LOs*' (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and '*product-related skills/LOs*'

Note: The weightage for the *process-related skills* and *product-related skills* will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	Correctly creating and running MATLAB scripts (coding hygiene, commenting, workspace handling).	6	
2	Selecting appropriate arithmetic operations for the given data.	5	
3	Following safety, coding ethics, and good laboratory practices (file management, backups).	4	
4	Contribution as a team member (peer collaboration, sharing datasets, debugging).	3	
Product-related Skills: 12 Marks - 40%			
1	Producing accurate computation results and well-formatted MATLAB Table output.	3	
2	Correct interpretation of logical comparisons.	2	
3	Interpretation of results (clarity, mathematical validity).	3	
4	Drawing meaningful conclusions using NEP-aligned verbs (justify, determine, generate).	2	
5	Ability to answer practicum-related higher-order questions.	1	
6	Submitting the report on time with screenshots and code	1	
Total		30	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	NEAVIS RISHVA. J	URK25CS1236

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	Product-Related Skills (12)	Total (30)		
